Version 2.0

January 2026

DATABASE

MODULE 1 - BASIC CONCEPT OF DATABASES

Author:

Dr. Ir. Agus Eko Minarno, M.Kom., IPM.

Naufal Arkaan

Ismail Dwi Muh. Anugerah

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

INTRODUCTION

OBJECTIVES

1. Students understand the fundamental principles of database systems and Database Management Systems (DBMS).
2. Students understand data modeling concepts, specifically Entity Relationship Diagrams (ERD) using Barker Notation.
3. Students understand how to identify entities, attributes, and data types for database design.

MODULE TARGETS

1. Students are able to distinguish between Conceptual Data Models (CDM) and Physical Data Models (PDM).
2. Students are able to design a Conceptual Data Model (CDM) using Oracle SQL Developer Data Modeler.
3. Students can correctly classify attributes (Mandatory/Optional, Single/Composite) and assign appropriate data types.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson.

   رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop or personal computer.
3. **Oracle SQL Developer Data Modeler** installed and ready to use.
   LINK-SQL_Developer_Data_Modeler
4. Strong motivation and commitment to learning.

KEYWORDS

Database, DBMS, Entity, Attribute, ERD, CDM, PDM, Oracle Data Modeler.

BASIC CONCEPT OF DATABASES

A database is an organized collection of data stored in a computer so it can be easily accessed, managed, and updated. A database is usually managed by a special software system called a Database Management System (DBMS), such as MySQL, PostgreSQL, Oracle, SQL Server, or MongoDB. The DBMS acts as an interface between users/applications and the stored data. It ensures that data can be retrieved quickly, kept consistent, protected from unauthorized access, and safely stored even when many users access it at the same time.

1. DATABASE AND DBMS

Imagine a large, well-organized cabinet where you keep all your important files and records. That cabinet is like a database. A Database is a **centralized** and **structured** collection of data stored in a computer system. Centralized means the data is kept in one main place. Structured means the data is organized neatly, usually in tables. The DBMS is the tool that helps you manage your files. It provides key functions when you need them:

1. **Retrieve:** Find and read data (e.g., searching for a student's grade).
2. **Add:** Insert new data (e.g., registering a new student).
3. **Modify:** Update existing data (e.g., changing a student's address).
4. **Delete:** Remove data (e.g., removing a course that is no longer offered).

A DBMS helps transform data into useful information (e.g., calculating the average GPA of all students).

2. DATA MODELING AND ERD

Before building a house, you need a blueprint. Before building a database, you need a Model. The process of designing this database blueprint is called Data Modeling. We use a special diagram called an Entity Relationship Diagram (ERD) to visualize this model. In this course, we will use Barker Notation for our ERD. Building an ERD starts with understanding the basic notations and terms in the Relational Model (how data connects). The design can be created using tools such as Oracle SQL Developer Data Modeler.

3. RELATIONAL DATABASE

The Relational Model is the most common way to structure data. A relational database organizes information into tables (like spreadsheets). This system allows you to access data efficiently without rebuilding tables every time.

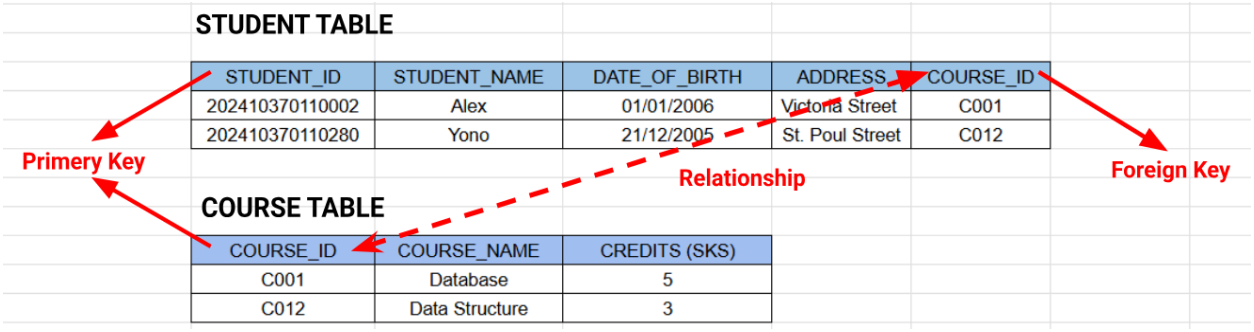


Figure 1. Example of a Relational Table

Keys are crucial for organization:

1. Primary Key: A **unique** identifier for every row in a table.
2. Foreign Key: A column in one table that **links to the Primary Key in another table**.

ENTITY AND ATTRIBUTE

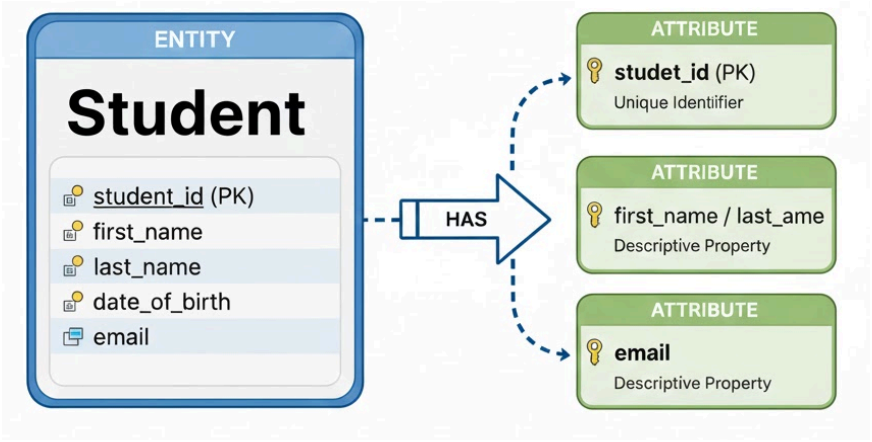


Figure 2. Entity and Attributes

1. ENTITY

An entity is a real-world object or concept that is important to the system and is represented as a table in a database.

TYPES OF ENTITY

NAME	DESCRIPTION
Prime	Independent
Characteristic	Formed due to another entity (prime).
Intersection	Formed due to two or more entities.

Table 1. Types of Entity

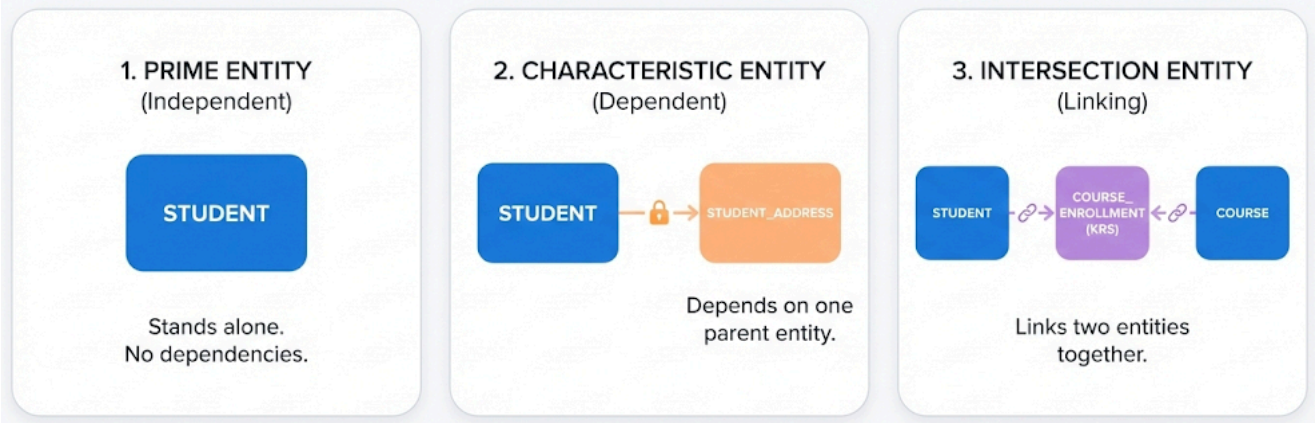


Figure 3. Types of Entity

EXPLANATION

- **Prime entity:** An entity that stands on its own. This entity can be created even if no other entity exists before it. For example, in a database model for course management, the Student entity can be created directly without depending on any other entity.
- **Characteristic entity:** An entity that cannot stand on its own. This entity exists as a characteristic of another entity. For example, in a database model for course management, the entity Student Address exists because there is a Student entity. The Student Address entity cannot be created without the Student entity first.

- **Intersection entity:** An entity that exists because of a many-to-many relationship between two entities. When two entities have a many-to-many relationship, an intersection entity must be added in the middle with one-to-many relationships to both original entities. The many side is on the intersection entity, and the one side is on the original entities. For example, in a course management database model, the entity Course Enrollment is an intersection entity between Student and Course. This intersection entity exists because the original relationship between Student and Course is many-to-many.

2. ATTRIBUTE

A set of information that is closely related to and describes an entity; known as the properties of an entity.

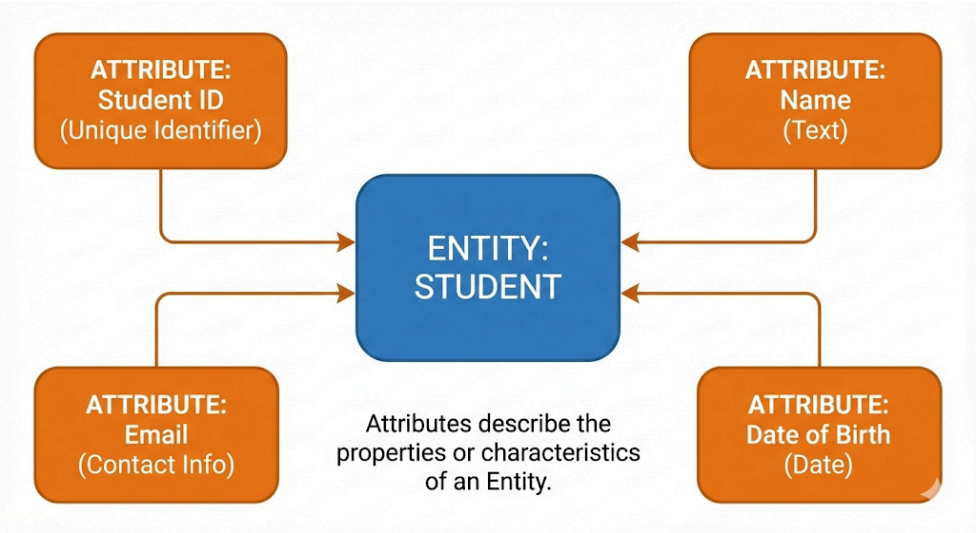


Figure 4. Attribute

ATTRIBUTE CLASSIFICATION

- **Mandatory & Optional**
  - ➢ Mandatory: cannot be empty or null, marked with *.
  - ➢ Optional: can be empty or null, marked with o.

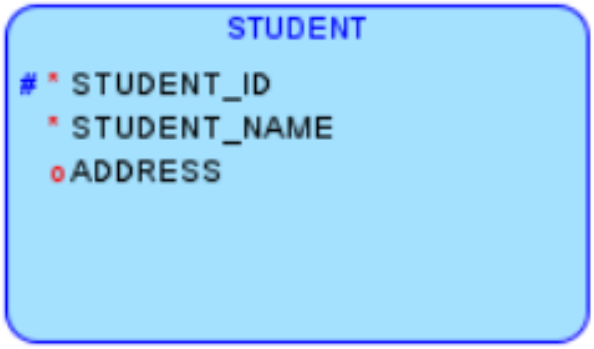


Figure 5. Example of Mandatory and Optional Attribute

- **Volatile & Non-Volatile**
  - ➢ Volatile attribute: unstable attribute. Example: age. Age is volatile because its **value changes over time**.
  - ➢ Non-volatile attribute: stable attribute. Example: date of birth. The date of birth is non-volatile because once recorded, the **value does not change**.

- **Single & Composite**
  - ➢ Single attribute: attribute with no smaller parts. Example: age. Age is a single value and cannot be broken down further.
  - ➢ Composite attribute: an attribute with smaller parts that have their own meaning. Example: full name. The full name is composed of first name, middle name, and last name.

- **Single-value & Multi-value**
  - ➢ Single-value attribute: an attribute with only one value at a time. Example: student ID. Each student has one unique ID during their study period.
  - ➢ Multi-value attribute: an attribute with more than one value at a time. Example: address. A person can have multiple addresses, such as home address and dorm address.

An attribute can be composite, multi-value, or both, depending on the design.

DATA TYPES

1. DEFINITION

In a database, every attribute must have a data type. A data type defines what kind of data can be stored, how much space is allocated, and what operations are allowed on that data.

At this point, it is important to understand that data types in databases are conceptually different from data types in programming languages.

- In a programming context, data types are used to:
  - ➢ control how data is processed in memory,
  - ➢ define how variables behave during program execution,
  - ➢ support calculations, conditions, and logic inside an application.

Examples of programming data types include int, float, string, or boolean.

- In a database context, data types are used to:
  - ➢ define how data is stored permanently,
  - ➢ enforce data integrity and constraints,
  - ➢ control how data can be queried, compared, and indexed.

This means that database data types focus more on storage rules and consistency, rather than program logic.

2. DATA TYPES IN DATABASE VS PROGRAMMING

Although some data types appear similar in name, their purposes are different.

- In programming:
  - ➢ data types describe how a variable behaves during execution,
  - ➢ values are temporary and exist only while the program is running.

- In databases:
  - ➢ data types describe how data is stored and validated,
  - ➢ values are persistent and remain stored even after the program ends.

- For example, an INTEGER in a database is not just used for calculations, but also:
  - ➢ defines valid value ranges,
  - ➢ affects indexing and performance,
  - ➢ determines how data is physically stored.

Because of this difference, database data types must be chosen carefully based on the nature of the data, not just based on familiarity with programming languages.

3. COMMON DATA TYPES

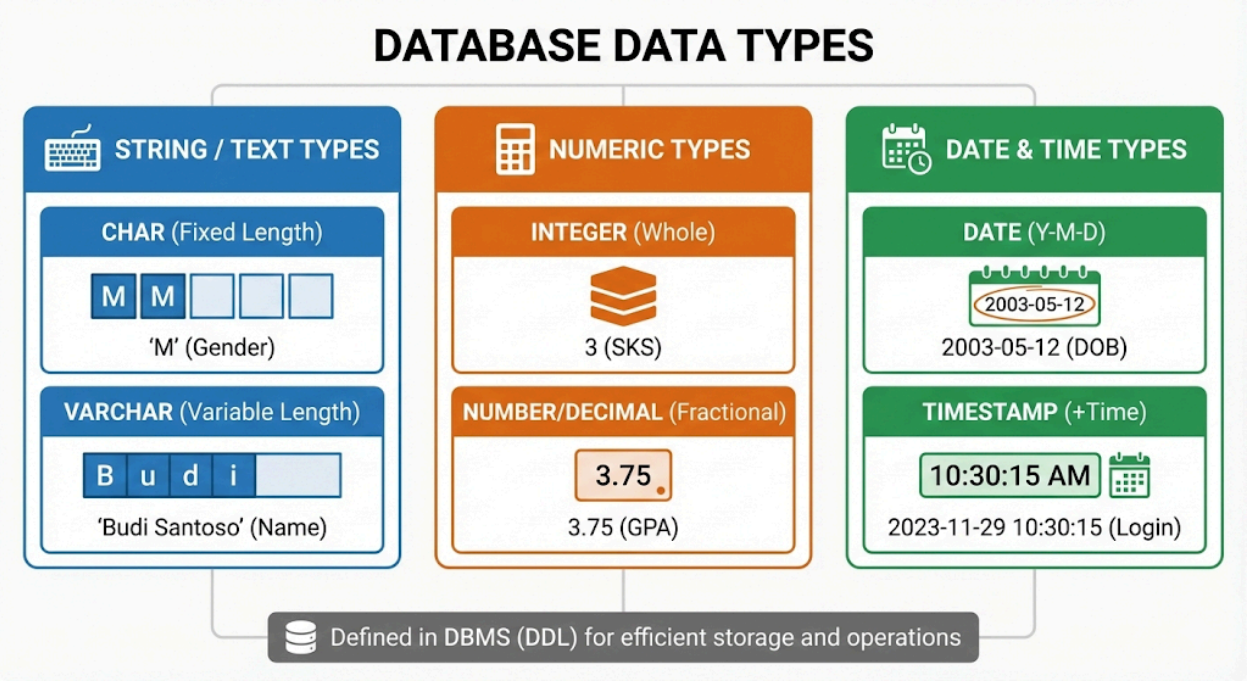


Figure 6. Common Data Types in Database

STRING OR TEXT TYPES

Used for letters, words, sentences, or alphanumeric codes (like ID numbers that start with letters).

- **CHAR (Fixed Length Character)**

  CHAR stores text of a fixed length. If you define it as CHAR(10) and only type “Hello” (5 letters), the database adds 5 spaces to fill it up to 10. Use CHAR for data that is always the same length.

- **VARCHAR (Variable Character)**

  VARCHAR stores text of variable length. If you define it as `VARCHAR(255)` and type "Budi", it only uses space for 4 letters. It saves storage space. Use VARCHAR for data where the length changes from person to person.

NUMERIC TYPES

Used for math calculations or counting.

- **INTEGER**

  INTEGER stores whole numbers (no decimals). Use INTEGER for things you can count or specific IDs that are only numbers.

- **NUMBER, FLOAT, AND DECIMAL**

  NUMBER, FLOAT, and DECIMAL store numbers that allow decimals (fractions). Use those data types for precise measurements or averages.

- **PRECISION AND SCALE**

  Precision refers to the total number of digits that a numeric value can contain, while scale refers to the number of digits to the right of the decimal point.

  Precision and scale are important in database design because incorrect numeric definitions can lead to data truncation or rejection. These concepts are commonly applied when storing financial data, measurements, or calculated values.

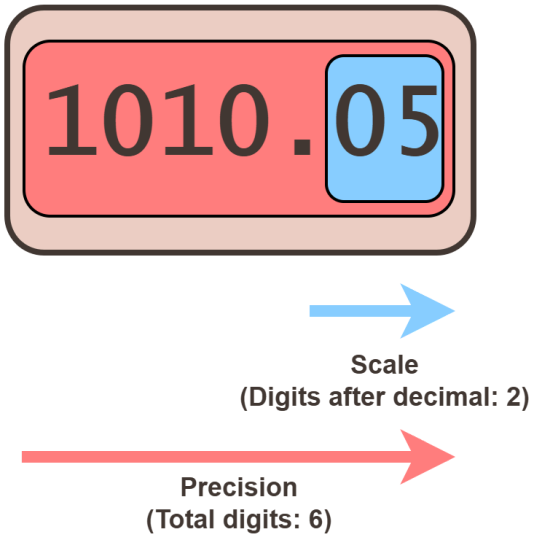


Figure 7. Example of Precision and Scale

DATE AND TIME TYPES

Used for scheduling and recording history.

- **DATE**

  DATE stores the year, month, and day.

- **TIME**

  TIME stores the precise time (hour, minute, second).

CONCEPTUAL DATA MODEL (CDM) AND PHYSICAL DATA MODEL (PDM)

1. CONCEPTUAL DATA MODEL (CDM)

A CDM represents entities and relationships without data types, showing how tables relate to each other for database implementation. A valid CDM can be converted to a PDM. Example:

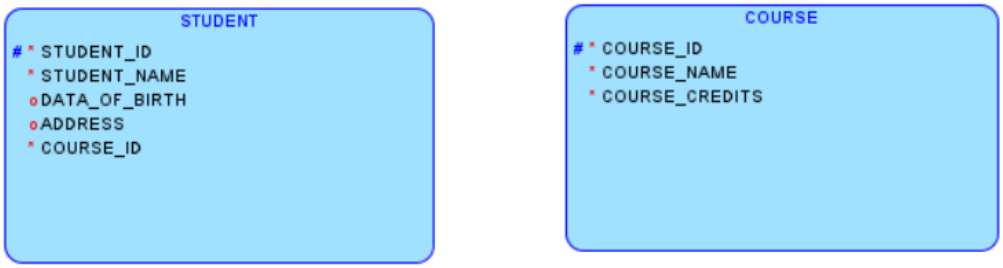


Figure 8. Example of CDM

2. PHYSICAL DATA MODEL (PDM)

A PDM is a more detailed concept that defines tables, data, data types, and relationships between tables. A PDM is the physical design of a database that is ready for implementation in a DBMS. It shows all table structures, including columns, primary keys, and foreign keys. Example:

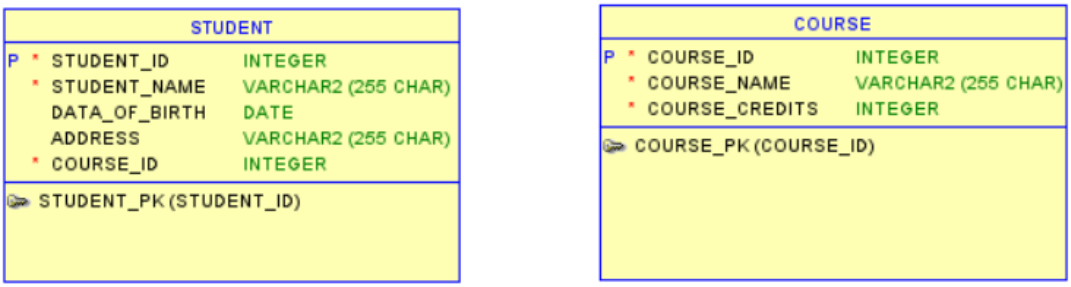
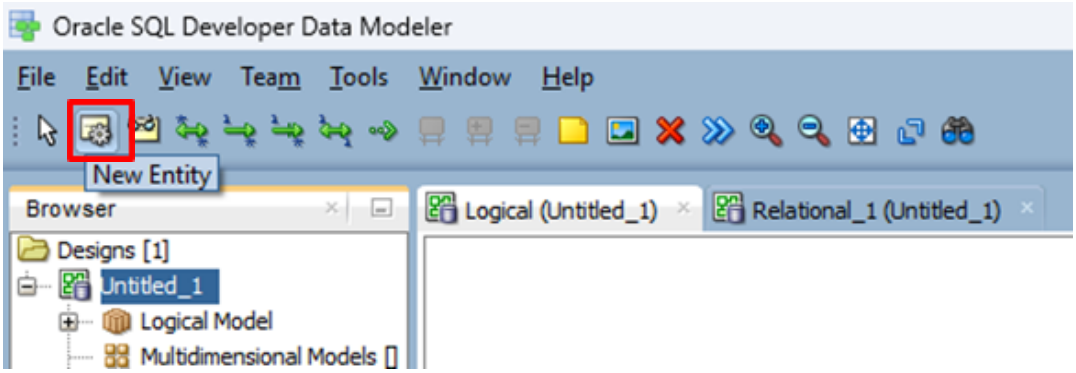


Figure 9. Example of PDM

PRACTICE ACTIVITY

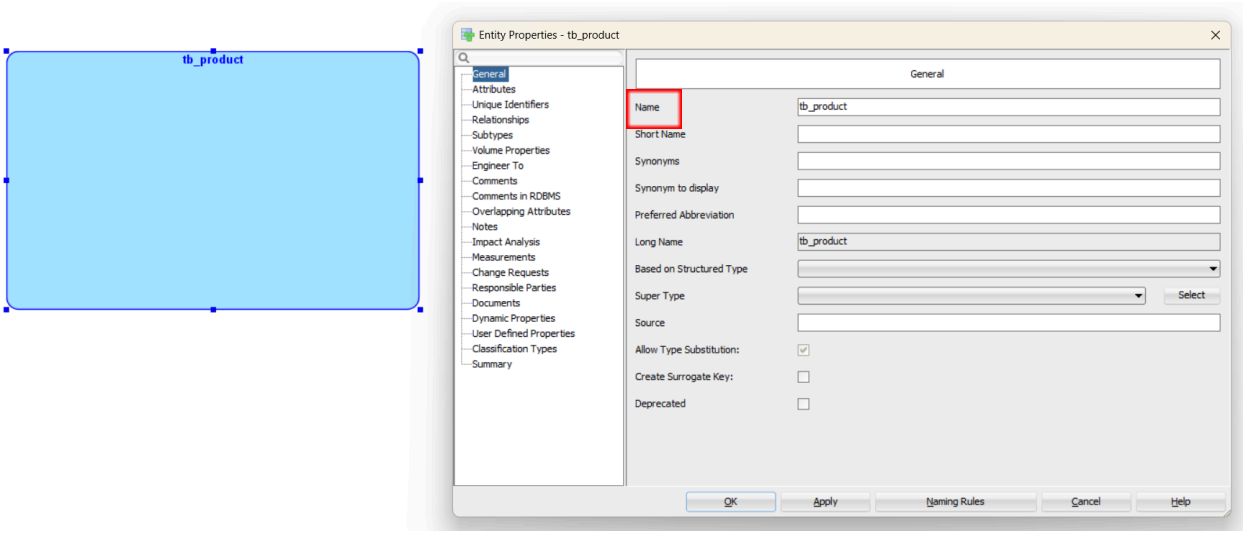
TUTORIAL FOR CREATING A CONCEPTUAL DATA MODEL (CDM)

1. Download and install SQL Developer Data Modeler from LINK-SQL_Developer_Data_Modeler and match it with the operating system you are using.
2. After completing the installation correctly, run the application named **"Data Modeler"** from the extracted folder of the uploaded zip file.
3. Make sure you are on the **"Logical (Untitled_1)"** workspace.
4. Create the required entities by clicking the following icon (New Entity):

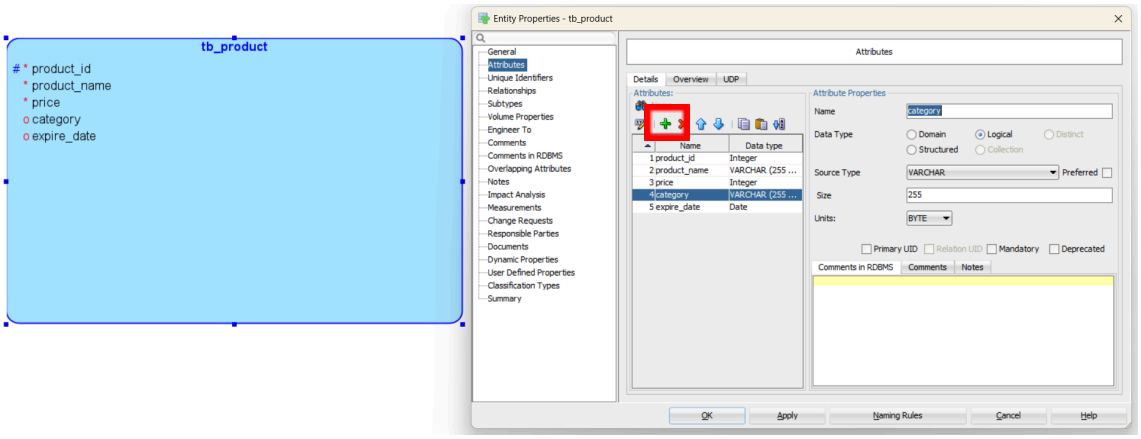


5. Move the cursor to the workspace and create the entity (similar to drawing a shape in Microsoft Word). After creating the entity, the "Entity Properties" window will appear as shown in the image.

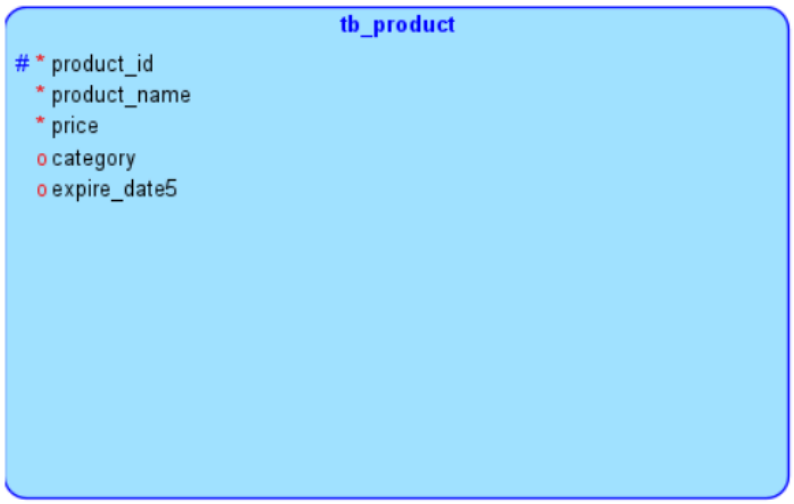
6. Click on the **“General”** menu. Set the name of the entity in the **“Name”** field. Click **“Apply”**, then click **“OK”**.



7. Switch to the **“Attributes”** menu. Click the **green plus (+)** button to add a new attribute. Set the attribute name by filling in the **“Name”** field. Change the **“Data Type”** from **“Domain”** to **“Logical”**. Set the **“Source Type”** according to the attribute you are creating. For an ID, you can use *Integer*. For attributes using *VARCHAR*, you can fill in the **“Size”** as needed, and for **“Units”**, you can choose between **BYTE** or **CHAR**. Lastly, for Primary UIDs, select the **“Primary UID”** checkbox to automatically mark it as mandatory. If an attribute is not a Primary UID but is required, select **“Mandatory”**. For optional attributes, simply leave the boxes unchecked. Click **“Apply”**.



8. If you want to add another attribute, click the **green plus (+)** button again and repeat the steps in point number 7. If you are done, click **"OK"**, and the entity creation is complete.



TRY OUT

In this **Try Out** section, you are required to add a minimum of 4 additional entities, each complete with attributes, based on an **Online Shop** theme. Each entity must have a minimum of 4 attributes, including a primary key, mandatory attributes, and optional attributes. Your final model must include the existing `tb_product` as well as your 4 new entities. The Try Out prepares you for the individual Assignment, where stricter constraints and assessment criteria apply.

Note: If you are unsure how to create an entity, feel free to ask the assistants.

ASSIGNMENT

Create a Conceptual Data Model (CDM) individually based on the scheme you have filled in the spreadsheet below.

ADD_YOUR_SCHEMA_HERE

NB: The CDM you design in this assignment will be used continuously until Module 6. Therefore, make sure that your schema is consistent, logical, and complies with all requirements on the spreadsheet.

ASSIGNMENT REQUIREMENTS

1. Design a Conceptual Data Model (CDM) consisting of at least five (5) entities, including exactly one (1) prime entity.
2. Each entity must contain at least four (4) attributes, consisting of:
   - at least two (2) Mandatory (*) attributes, and
   - at least two (2) Optional (o) attributes.
3. For each attribute in the CDM, you must use the common use data types at least two (2) (string or text, numeric, date and time).
4. For each attribute in the CDM, you must:
   - assign Mandatory (*) or Optional (o) marking, and
   - classify the attribute as Volatile (V) or Non-Volatile (NV).
5. All attribute markings and classifications must be logically consistent with the nature of the data.

ASSESSMENT

You must explain this assessment to the assistant during the demo week.

1. Based on your CDM, mention the types of entities you used (prime, characteristic, or intersection). Choose one entity and explain why it belongs to that type.
2. Explain the difference between a composite attribute and a multi-value attribute. Give one example of each from your CDM.
3. Choose one mandatory attribute and one optional attribute from your CDM. Explain why each attribute is marked that way.
4. Choose one volatile attribute and one non-volatile attribute from your CDM. Explain why each attribute is classified as volatile or non-volatile.
5. Explain the difference between data types in a programming context and data types in a database context. Based on your CDM, mention one example of a database data type and explain its purpose in database design.

PRACTICUM GRADING RUBRIC

Assessment Aspects	Points
Try Out	**Total 5%**
• 4 new entities are added. • Each entity has at least 4 attributes. • For each entity, at least 1 mandatory (*) attribute and at least 2 optional (o) attributes are defined. • Entity and attribute names are relevant to the schema.	100
• 4 new entities are added. • Each entity has at least 3 attributes. • For each entity: at least 1 mandatory (*) attribute, but optional (o) attributes are fewer than 2.	80
• 4 new entities are added. • One entity has at least 3 attributes, the other has only 2 attributes. • Mandatory or optional markings are not applied to all attributes.	70
• Only 2 new entities are added. • The entity has at least 2 attributes, but mandatory or optional marking is missing.	60
• An entity is added with only 1 attribute, or • Attributes exist but no mandatory (*) or optional (o) marking is applied.	40
• No new entity is added, or • The submission does not follow the Try Out instructions.	0
Assignment	**Total 30%**
• 5 entities are created. • Exactly 1 prime entity is clearly identified. • Each entity has at least 3 attributes (1 Mandatory + 2 Optional). • Mandatory (*) and Optional (o) markings are applied to all attributes. • Volatile / Non-Volatile classification is applied to all attributes and is logical.	100

• 5 entities are created. • Prime entities exist but are not clearly justified. • One entity does not fully meet the attribute composition requirement. • Volatile / Non-Volatile classification is applied, but some attributes are incorrect. • Applied common use data types (String or text, numeric, date and time)	80
• 4–5 entities are created. • Prime entity is unclear or inconsistently defined. • The attribute composition requirement is met in only some entities. • Volatile / Non-Volatile classification is incomplete.	70
• Fewer than 5 entities are created. • The prime entity is not clearly identified. • Mandatory / Optional marking is partially missing. • Volatile / Non-Volatile classification is mostly missing or incorrect.	60
• Entities are created, but most entities have fewer than 3 attributes. • Mandatory / Optional marking is rarely applied. • Volatile / Non-Volatile classification is not applied.	40
• No valid CDM is submitted. • The submission does not follow the Assignment instructions.	0
Assessment	**Total 65%**
• Correctly identifies entity types used in the CDM. • Clearly explains why one chosen entity is classified as prime, characteristic, or intersection. • Correctly explains the difference between composite and multi-valued attributes and provides correct examples from the CDM. • Correctly explains mandatory and optional attributes using examples from the CDM. • Correctly explains volatile and non-volatile attributes using examples from the CDM. • Correctly explains the difference between data types in programming and data types in a database, and provides one correct database data type example from the CDM. • All answers are consistent with the student's CDM.	100
• Identifies entity types used in the CDM. • Explains entity classification, but the explanation is incomplete. • Correctly explains composite and multi-valued attributes, but examples are incomplete or not from the CDM. • Explains mandatory and optional attributes, but one example is	80

incorrect. • Explains volatile and non-volatile attributes, but one classification is incorrect. • Explains data type differences, but the example is incomplete.	
• Identifies entity types but cannot clearly justify classification. • Can mention composite and multi-valued attributes but mixes the examples. • Can mention mandatory and optional attributes but cannot explain the reason clearly. • Can mention volatile and non-volatile attributes but misclassifies some attributes. • Mentions data types but confuses programming and database context.	70
• Cannot correctly identify entity types. • Cannot distinguish composite and multi-valued attributes. • Mandatory and optional attributes are mostly incorrect. • Volatile and non-volatile classification is incorrect. • Does not understand the difference between programming and database data types.	60
• Can answer only one or two questions correctly. • Most explanations are not related to the CDM. • Examples are missing or incorrect.	40
• Does not answer the assessment questions, OR • Answers are not related to the CDM, OR • The student is absent during the assessment.	0

NB:

1. **If a student attends the material session and receives a score for the practicum activities, but is unable to fully answer the assessment questions during the demo week, the student must still be given a minimum assessment score of 40.**
2. **If a student does not attend the material session, does not complete the Try Out, receives a score of 0 for the Assignment, and cannot answer the assessment questions at all, then the assessment score must be 0.**